

Artificial Intelligence in Software Testing: A Literature Search

Grace Shevchuk
Software Engineering
University of Maryland,
Baltimore County

Abstract: Software testing is an important, although time consuming, part of the software development lifecycle. Early and efficient software testing is able to significantly cut costs for developers and provide higher-quality products for users. This paper examines recent literature focused on integrating Artificial Intelligence and Machine Learning into software testing. The focus of this paper was to determine what benefits AI has in terms of software testing, if it is effective, and if wide-scale implementation is possible. Recent research shows that AI is able to significantly cut costs and time during software testing by automating tests, generating tests, prioritizing tests, and refactoring code, although there are some issues, like challenges with data requirements needed for the models used. While AI implementation in software testing is beneficial to developers, more research is needed to make wide-scale implementation possible.

I. INTRODUCTION

Software testing is a necessary part of the software development process, although it is not always given the priority it needs due to its time consuming and expensive nature. Software testing can be broken up into four general categories: test design, test automation, test execution, and test evaluation. These categories range in terms of technical background, theory knowledge, and computer skills required to complete, although a test developer may be assigned tasks from each of these categories. This can create inefficient and mundane work. Due to the importance of software testing and its challenges, methods to improve the process is a priority to ensuring high quality programs from users at lower prices for developers.

With the growing interest in Artificial Intelligence (AI) and Machine Learning (ML) in the past decade, there has been a rise in the number of studies focused on using AI and ML during the software development process, and in this paper I will focus specifically on software testing. With AI, there is the potential of significantly cutting the cost and time needed to complete software testing, as well as allowing test developers to focus on the categories of software testing that require more technical background and theory knowledge, like test evaluation.

In this paper, I conduct a literature search of research from the past five years focused on implementing AI during software testing. This paper aims to find what benefits AI has when used during software testing, if these benefits outweigh any potential costs, and if wide-scale implementation is possible. This paper will also look at any open issues and directions for future research.

II. METHODOLOGY

My approach to this literature search was to find a broad range of literature on my chosen topic, then to narrow based on articles that had varied, but relevant, focuses. This way, the literature chosen for this paper would, combined, give a full picture of recent research focusing on AI uses in software testing. IEEE Xplore was selected as the primary library used, with a majority of articles found through this library. The IEEE Computer Society Digital Library and Science Direct were also used to search for articles. All searches were filtered for articles published between 2021-present to get the most recent and relevant research on the topic.

A. Search Terms

"Artificial intelligence software testing" was the first search term used. This yielded three papers that discussed the use of AI in software testing. All three of these papers were kept for the final selection, as they provided specific implementation of AI in some aspect of software testing.

From there, "AI software development" was used as a search term. This broadened the range of articles, with nine articles having a significant section focused on software testing. From these nine, five were kept for the final selection, three of which were literature reviews that provided broader insights into AI implementation in software testing. The oldest articles that contained repeating information were eliminated from the pool of literature.

The final search term used was "artificial intelligence refactoring." This was a more focused search aiming to cover an important part of the software testing process, which the current pool of papers found have not covered. This resulted in two articles being found, both of which were kept in the final selection. Throughout the searches, I alternated between using "artificial intelligence" and "AI" in attempts of getting more unique results between each search.

III. SUMMARY OF RESEARCH

AI tools can provide a lot of benefits to the software engineering process [1]. This paper will focus on the use of AI specifically in the testing phase of the software engineering lifecycle, looking at how recent studies have used AI during testing, what models were used, and any issues related to implementing AI in testing that have started to be addressed in research. Even though wide-spread implementation of AI tools during software testing, and during software development in

general, is just beginning, AI is already playing a big role in software testing. Based on a Stack Overflow industry survey of 90,000 developers, 49% reported using Generative AI to help with the debugging process [2].

In testing, AI tools can help with regression testing by learning the entire test suite, reaching all parts of the software system. This will help developers check to see if any changes have caused faults in the program. AI tools can also automatically debug code after running the tests, as well as can optimize the code without changing its functionality, which is called refactoring [1]. AI can also be used to generate test cases and data, predict test effectiveness, and execute tests [2]. Testing can be slow and expensive, so using AI tools to prioritize the tests most likely to fail will lower the cost while giving developers feedback quickly [3]. A majority of studies looked at in a literature review of AI in the software development process focused on error prediction and a large number looked at verification and validation. Not as many studies have looked at test case and data generation, test case prioritization, and testing effort estimation [4].

AI tools can also greatly benefit testing for specific technologies and softwares. Take a look at two studies that looked at implementing AI into specific testing: penetration testing and microelectronics testing. Penetration testing is a necessary part of the software development lifecycle for web applications, ensuring security and privacy. However, penetration testing is very time-consuming [5]. Microelectronics have become an integral part of society, but they are extremely complex, making microelectronic testing face challenges such as high complexity, time-consumption, and the need for expert intervention [6]. Both of these studies have found that AI tools can provide solutions to these challenges faced by these critical testing processes. For penetration testing, AI can not only speed up testing times but also improve results by being used during the scanning, enumeration, and exploitation stages [5]. In microelectronic testing, ML algorithms, deep learning models, and expert systems have been found to improve the efficiency and accuracy. ML algorithms can be used to identify and predict potential faults, deep learning models can be used to improve precision of diagnosis, and expert systems can be used for automatic fault detection and repair decisions [6].

Data centers use large amount of energy and water, which is a major environmental concern. However, there has been little research on the environmental sustainability of AI integration during software testing. Recently, research has shown that AI usage in software testing improves sustainability, but its benefits may be outweighed since AI-generated tests can take time to interpret and may not be used [10].

A. Refactoring

Refactoring is an important process in the software development lifecycle, used for improving code structure, maintainability, and reusability without modifying the code's functionality [7]. Refactoring impacts code quality, performance, and energy consumption, however random refactoring can lower the quality of the system design and code, and the refactoring process cannot easily be automated. Design and code smells are predictors for future errors in the code, so refactoring these smells is important. A recent study found a

solution for integrating AI into the refactoring process by using a multi-objective optimization algorithm to prioritize parts of the program that need refactoring. The algorithm used was able to find the best improvements to the software while also removing all the design smells [8].

B. Explainable AI

The use of AI in software engineering has been increasing, however the use of "black-box" models provides a limitation to full implementation. A common issue faced when implementing AI during software testing is understanding why a test failed. This issue can be solved by Explainable Artificial Intelligence (XAI) [9]. Another major challenge to using AI during software testing is the time it takes to interpret tests and results, and XAI implementation can help mitigate that [10]. A third aspect of software testing that XAI can benefit is when using AI for error prediction and refactoring suggestions. Developers need to be certain that these suggestions are accurate and will not cause more errors, so using XAI will help developers review suggestions and predictions. LIME/SHAP and counterfactuals are the most common tools used in literature when using XAI for software testing [9].

C. Commonly Used Models

The vast majority of models used with software testing in research are supervised learning techniques, with ANN, random forest, SVM, decision tree, and naive bayes commonly looked at as well [4]. Supervised learning is also particularly effective with penetration testing [5]. Additionally, ML models are commonly used when integrating AI into the software testing process, focusing on saving developers time and money [3]. ML methods are also more suited for refactoring prediction and prioritization. For other aspects of refactoring, random forests and genetic algorithms are commonly used for optimization, and support vector machines, convolutional neural networks, and decision trees are commonly used for versatility and effectiveness in recent literature [7].

IV. SUMMARY OF OPEN PROBLEMS

One of the biggest issues currently faced when researching implementations of AI into software testing is the data requirements. Most implementations require large data sets that are not easily available. For example, AI methods for penetration testing rely on high-quality annotated data, which requires significant manual effort [5].

Additionally, with the increased interest in using AI during the software development process and software testing, there has been a large increase in the amount of research on the topic. This large spike in literature on AI usage during software testing can lead to an oversaturation of results, making it difficult to identify the most impactful findings [2]. This will not only make future research more difficult, but also make implementation for developers more difficult.

Another major issue research faces is challenges with reproducibility of results. Most of the models used in research are "black-box," meaning that the steps the AI model takes to make decisions are not known. In addition to creating challenges in the reproducibility of results of prior research, the use of "black-box" models causes findings to be limited by

generalizability, since overfitting is a major limiting factor [4]. Even though XAI is an available solution for this issue moving forward, it is not able to retroactively fix the current challenges with reproducibility.

V. CONCLUSIONS

This paper conducted a literature search of recent research focusing on Artificial Intelligence usage in software testing. Beginning with 14 papers from the last five years, I narrowed down the pool of literature to 10 papers to get a varied set of studies. Generally, AI usage in software testing provides many benefits. It is able to greatly reduce testing time and automate parts of the testing process.

Specifically, AI is able to be used for debugging code, generating tests, prioritizing tests, executing tests, refactoring, and predicting errors. Supervised learning techniques and ML models are most commonly used when integrating AI into the testing process. When AI is implemented into the testing process, it is able to save developers time and money, and also complete the tasks efficiently and effectively.

While there has been a large increase in research focusing on AI usage in software testing, there are still some open issues. One issue is caused by this large increase, which is an oversaturation of results, making it more difficult to identify the most significant findings. Another issue is the difficulty to find usable data sets for research, which makes significant manual effort required. A third issue is challenges with reproducibility, since most of the models used in research are "black-box." These open issues lead to potential avenues for future research.

A. Future Work

To address the issue currently faced with generalizability, researchers have suggested devising a feedback mechanism [4]. Also, to address the issue of reproducibility, research moving forward can use XAI models. This will not fix issues with past papers, but it will help mitigate this issue moving forward. More research focusing on validation, evaluation, and experience is also needed to make AI integration into software development possible [7].

Despite the difficulties with finding large datasets, more experiments using larger numbers of datasets and software applications is recommended [4]. Also, there is a current gap in

research focusing on model performance statistics and overviews of best methods for integrating AI into the software testing process [7].

REFERENCES

- [1] Mashkoo, A., Menzies, T., Egyed, A., & Ramler, R. (2022). Artificial Intelligence and Software Engineering: Are we ready? *Computer*, 55(3), 24–28. <https://doi.org/10.1109/mc.2022.3144805>.
- [2] Bannon, T. (Trac). (2024). Infusing artificial intelligence into software engineering and the devsecops continuum. *Computer*, 57(9), 140–148. <https://doi.org/10.1109/mc.2024.3423108>.
- [3] Jha, N., & Popli, R. (2021). Artificial Intelligence for software testing-perspectives and practices. *2021 Fourth International Conference on Computational Intelligence and Communication Technologies (CCICT)*, 377–382. <https://doi.org/10.1109/ccict53244.2021.00075>.
- [4] S. Shafiq, A. Mashkoo, C. Mayr-Dorn and A. Egyed, "A Literature Review of Using Machine Learning in Software Development Life Cycle Stages," in *IEEE Access*, vol. 9, pp. 140896-140920, 2021, doi: 10.1109/ACCESS.2021.3119746.
- [5] G. Sánchez, O. Olayinka and A. Pasikhani, "Web Application Penetration Testing with Artificial Intelligence: A Systematic Review," *2024 22nd International Symposium on Network Computing and Applications (NCA)*, Bertinoro, Italy, 2024, pp. 236-245, doi: 10.1109/NCA61908.2024.00043.
- [6] Y. Zhang, H. Liu and F. Xu, "Research on the Application of Artificial Intelligence Technology in Microelectronics Testing and Fault Diagnosis," *2024 3rd International Conference on Data Analytics, Computing and Artificial Intelligence (ICDACA)*, Zakopane, Poland, 2024, pp. 343-347, doi: 10.1109/ICDACA165086.2024.00070.
- [7] S. Motogna, L. -M. Berciu and V. -A. Moldovan, "Artificial Intelligence Methods in Software Refactoring: A Systematic Literature Review," *2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, Paris, France, 2024, pp. 309-316, doi: 10.1109/SEAA64295.2024.00055.
- [8] AbuHassan, A., Alshayeb, M., & Ghouti, L. (2022). Prioritization of model smell refactoring using a covariance matrix-based adaptive evolution algorithm. *Information and Software Technology*, 146, 106875. <https://doi.org/10.1016/j.infsof.2022.106875>.
- [9] Arora, L., Girija, S. S., Kapoor, S., Raj, A., Pradhan, D., & Shetgaonkar, A. (2025). Explainable artificial intelligence techniques for software development lifecycle: A phase-specific survey. *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2281–2288. <https://doi.org/10.1109/compsac65507.2025.00321>.
- [10] E. Trinh, M. Funke, P. Lago and J. Bogner, "Sustainability Integration of Artificial Intelligence into the Software Development Life Cycle," *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, Hyderabad, India, 2024, pp. 199-206, doi: 10.1109/ICSA-C63560.2024.00044.